Startup Success Prediction and Investment Recommendation Using Advanced Stacking Ensemble Learning

Suna Hera Akter Suna
22201168
Department of Computer Science and Engineering
University of Asia Pacific

MD. Sydul Islam Zubair
22201174
Department of Computer Science and Engineering
University of Asia Pacific

Faiza Haque Shoily
22201183
Department of Computer Science and Engineering
University of Asia Pacific

***Abstract*—Startup investment decisions are inherently risky due to limited historical data, subjective evaluation criteria, and complex market dynamics. This paper presents a machine learning system for predicting startup success and generating data-driven investment recommendations. Using the Kaggle Startup Success Prediction dataset (∼1,000 startups, 49 features), four baseline models were trained and evaluated: Logistic Regression, Decision Tree, Random Forest, and XGBoost. An Advanced Stacking Ensemble was subsequently developed, combining tuned Random Forest and XGBoost as base learners with Logistic Regression as the meta-learner. The proposed ensemble achieved the best overall performance: ROC-AUC of 0.847, Accuracy of 78.9%, and F1-Score of 0.845. Funding amount, investor relationships, and milestone completion were identified as the strongest predictors of startup success. The system outputs four-tier investment recommendations (Strong Invest, Moderate Invest, High Risk, Avoid) based on predicted success probabilities.**

***Index Terms*—Machine Learning, Startup Success Prediction, Stacking Ensemble, Investment Recommendation, Random Forest, XGBoost, Binary Classification**

I. Introduction

Early-stage startup investment is one of the most challenging decision-making environments in modern finance. Studies indicate that over 90% of startups fail within their first five years, making systematic evaluation tools critically important for investors and venture capital firms alike [1]. Traditional venture capital evaluation relies on investor intuition, personal networks, and qualitative assessments — approaches that are inherently subjective and difficult to scale.

Machine learning offers a compelling alternative by extracting predictive patterns from historical startup funding data. This paper proposes a hybrid ML system addressing startup success prediction as a binary classification problem: predicting whether a startup will be *acquired* (success) or *closed* (failure), and integrating the predictions into an actionable investment recommendation engine.

The key contributions of this work are:

- A rigorous data pipeline with systematic leakage removal from outcome-correlated features
- Domain-specific feature engineering: funding-per-round ratio, milestone efficiency, and investor scoring
- An Advanced Stacking Ensemble achieving F1-Score of 0.845 — best among all evaluated models
- A probability-based four-tier investment recommendation engine suitable for practical deployment

II. Related Work

Prior research has demonstrated that funding features and investor network centrality are strongly predictive of startup acquisition outcomes [2]. Logistic Regression has served as a standard interpretable baseline in startup prediction tasks [3]. Random Forest shows strong performance on structured tabular data with mixed feature types [4], while XGBoost excels on small-to-medium structured datasets [5]. Stacking ensemble methods, where base learner predictions serve as input to a meta-learner, demonstrate consistent performance gains over individual models [6]. However, application of stacking to startup prediction with systematic leakage prevention and domain-specific feature engineering remains underexplored in the literature.

III. Dataset and Preprocessing

A. Dataset Description

The Kaggle Startup Success Prediction dataset [7] contains ∼1,000 startup records with 49 original features sourced from Crunchbase. The binary target variable represents startup outcome: *Acquired* (1) for successful acquisition, and *Closed* (0) for ceased operations.

Key feature categories include: total funding amount (USD), number of funding rounds, investor relationship count, U.S. state code flags (CA, NY, MA, TX), industry category codes

(pre-encoded), milestone counts, and binary round participation indicators (`has_roundA` through `has_roundD`), plus VC and angel investor flags.

B. Data Cleaning

Identifier columns (`id`, `name`, `permalink`), administrative fields (`zip_code`, `city`), and temporal columns (`founded_at`, `first_funding_at`, `last_funding_at`, `closed_at`) were removed.

Critically, the `labels` column was identified as a direct source of **target leakage** — it encoded outcome-related information derived from the target variable. Its removal was essential for valid model evaluation. Missing values were imputed using median for numerical features and mode for categorical features.

C. Feature Engineering

Four derived features were engineered to capture latent business characteristics:

- `funding_per_round` = total_funding ÷ (rounds + 1): capital efficiency measure
- `milestone_efficiency` = milestones ÷ (relationships + 1): normalized milestone achievement
- `investor_score`: binary sum of has_VC, has_angel, has_roundA through has_roundD
- `log_funding_total_usd` = log(1+ total_funding): log-transform to normalize right-skewed distribution

D. Feature Importance

A preliminary Random Forest scan was run on the full feature set to identify the strongest predictors before model training. The top predictors, in order of importance, were: `relationships`, `age_last_milestone_year`, `log_funding_total_usd`, `milestone_efficiency`, and `funding_per_round` (Fig. 1).

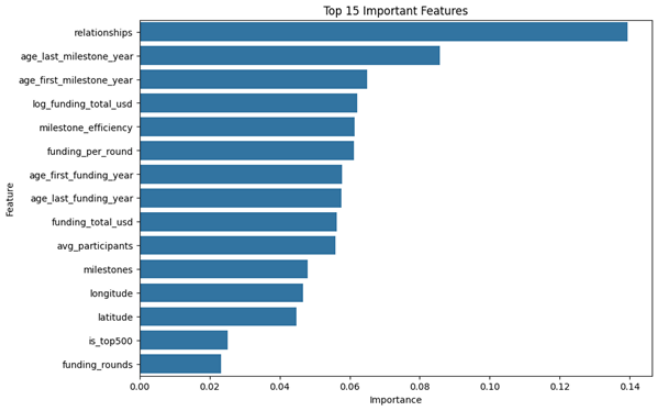

Fig. 1. Top 15 most important features identified by the Random Forest model. Investor relationships, milestone timing, and funding-related features dominate the predictive signal.

E. Train/Test Split and Scaling

The dataset was split 80%/20% using stratified sampling (`random_state=42`). `StandardScaler` was fitted exclusively on training data and applied to both splits to prevent data leakage.

IV. Methodology

A. Baseline Models

Four baseline classifiers were trained:

Logistic Regression: L2 regularization, `max_iter=500`, `class_weight='balanced'`. Serves as a linear baseline and subsequently as the stacking meta-learner.

Decision Tree: `max_depth=5`, `min_samples_split=10`. Interpretable non-linear baseline.

Random Forest: `n_estimators=300`, `max_depth=10`, `class_weight='balanced'`. Robust tabular baseline and first base learner.

XGBoost: `n_estimators=300`, `max_depth=6`, `learning_rate=0.05`, `subsample=0.8`. Strong gradient boosting baseline and second base learner.

B. Hyperparameter Tuning

`RandomizedSearchCV` with 5-fold CV and F1 scoring (20 iterations each) was applied to both RF and XGBoost.

RF search space: n_estimators ∈ [100, 200, 300, 500], max_depth ∈ [5, 10, 15, 20], min_samples_split ∈ [2, 5, 10].

XGBoost search space: n_estimators ∈ [100, 200, 300], max_depth ∈ [3, 5, 7], learning_rate ∈ [0.01, 0.05, 0.1], subsample ∈ [0.7, 0.8, 1.0].

C. Advanced Stacking Ensemble

Implemented using scikit-learn's `StackingClassifier`:

- **Base Learners:** Tuned Random Forest + Tuned XGBoost
- **Meta-Learner:** Logistic Regression on `predict_proba` outputs
- **CV:** 5-fold for out-of-fold base learner predictions
- **Passthrough=False:** meta-learner trained on base probabilities only

RF reduces variance via bagging; XGBoost reduces bias via boosting; the LR meta-learner discovers their optimal linear combination.

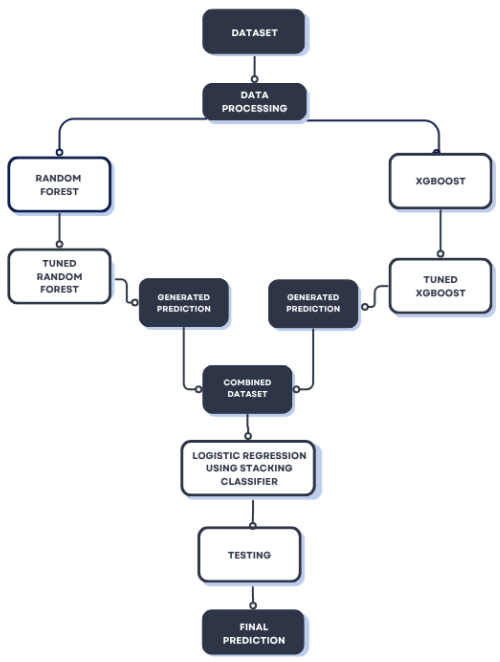

Fig. 2. Model architecture of the Advanced Stacking Ensemble: data flows through tuned Random Forest and tuned XGBoost base learners whose predictions are combined into a single dataset fed to a Logistic Regression meta-learner via the StackingClassifier, producing the final prediction.

V. Implementation and Results

A. Experimental Setup

All experiments were conducted in Google Colab (Python 3.10). Libraries: scikit-learn 1.3, XGBoost 2.0, Pandas 2.0, NumPy 1.24, Matplotlib 3.7, Seaborn 0.12 [8]. Models were evaluated on the held-out 20% test set (∼200 samples).

B. Investment Recommendation System

The stacking ensemble's predicted probability maps to a four-tier recommendation:

- **Strong Invest** ($p \geq 0.85$): high-confidence acquisition-likely startups
- **Moderate Invest** ($0.70 \leq p < 0.85$): good prospects with moderate uncertainty
- **High Risk** ($0.50 \leq p < 0.70$): uncertain — deeper due diligence required
- **Avoid** ($p < 0.50$): low predicted success probability

C. Model Performance Comparison

Table I presents the complete performance comparison across all five models on the held-out test set.

TABLE I
Performance Comparison of All Models on Test Set

Model	AUC	Acc	Prec	Rec	F1	RMSE
Logistic Reg.	0.784	0.703	0.842	0.667	0.744	0.545
Decision Tree	0.733	0.703	0.788	0.742	0.764	0.545
Random Forest	0.834	0.741	0.777	0.842	0.808	0.509
XGBoost	0.824	0.784	0.813	0.867	0.839	0.469
Adv. Stacking	**0.847**	**0.789**	0.809	0.883	**0.845**	**0.459**

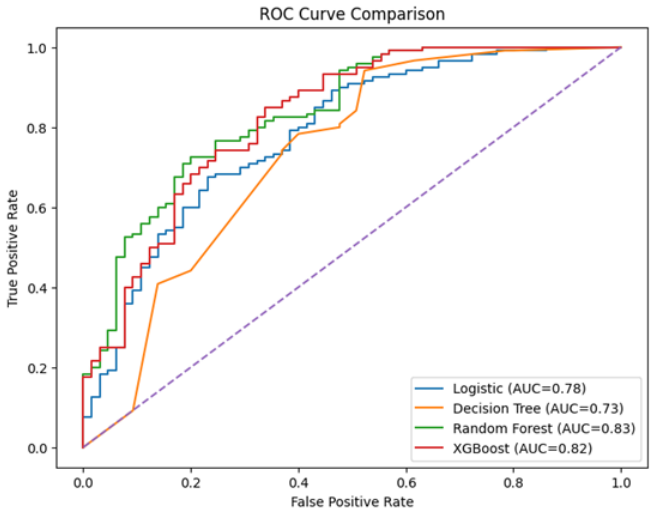

Fig. 3. ROC curve comparison across the four baseline models. Random Forest and XGBoost achieve the highest area under the curve, outperforming Logistic Regression and Decision Tree.

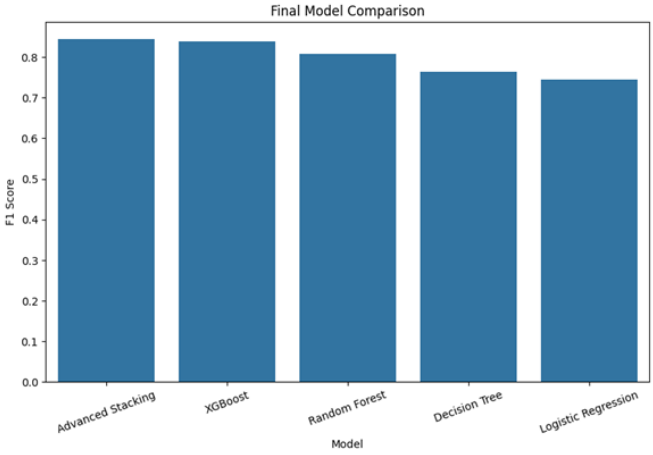

Fig. 4. Final F1-Score comparison across all five models, confirming the Advanced Stacking Ensemble as the top performer.

D. Key Findings

- Advanced Stacking Ensemble achieved best F1 (0.845) and ROC-AUC (0.847) among all models
- Random Forest achieved highest standalone ROC-AUC (0.834), confirming strength as base learner

- XGBoost achieved second-best F1 (0.839), providing complementary boosting signal
- Logistic Regression showed highest Precision (0.842) but lowest Recall (0.667) — conservative linear predictions
- These results are consistent with the feature importance ranking in Section III-D (Fig. 1) — relationship and milestone-timing features drive the strongest model performance

VI. Discussion

A. Model Performance Analysis

The results confirm that ensemble methods consistently outperform individual classifiers for this startup prediction task. The stacking architecture's advantage over Random Forest alone and XGBoost alone demonstrates the complementary nature of bagging (variance reduction) and boosting (bias reduction) when combined through a learned meta-learner.

B. Challenges and Limitations

Data Leakage: The `labels` column encoded outcome information directly. Detection and removal was the single most critical preprocessing step in the project.

Class Imbalance: Addressed through `class_weight='balanced'` consistently applied across all applicable models.

Dataset Scale: The ∼1,000 sample size limits generalization confidence. Future work should validate on larger, more recent Crunchbase exports.

VII. CEP Outcome Mapping

Table II maps this project to the Washington Accord Complex Engineering Problem (CEP) framework.

TABLE II
CEP Outcome Mapping (Washington Accord Framework)

Category	Codes	Contribution	Section
Knowledge Profile (WK)	WK1, WK2, WK3	Applied ML theory, statistical modeling, and ensemble learning to a real-world business prediction problem.	Sec. III
Complex Problem Solving (WP)	WP1, WP3, WP4	Formulated startup failure as a complex classification problem. Addressed data leakage and class imbalance.	Sec. IV
Engineering Activities (EA)	EA1–EA4	Designed full ML pipeline; resolved conflicting data issues; applied creative stacking ensemble with real-world investment consequences.	Sec. V
Programme Outcomes (PO)	PO1–PO3, PO5	Applied engineering knowledge, analyzed complex problem, designed ML solution, used modern tools: Python, scikit-learn, XGBoost.	All Secs.

VIII. Conclusion

This paper presented a complete ML framework for startup success prediction and investment recommendation. The Advanced Stacking Ensemble combining tuned Random Forest and XGBoost base learners with Logistic Regression as the meta-learner achieved the best performance: F1-Score of 0.845 and ROC-AUC of 0.847, outperforming all four baseline models. Careful data preprocessing — particularly leakage detection and removal — and domain-driven feature engineering were critical to these results.

Future work includes: (1) deep learning for sequential funding data; (2) real-time Crunchbase API integration; (3) SHAP explainability for investor transparency; (4) web-based deployment for non-technical users.

References

[1] S. Blank and B. Dorf, *The Startup Owner's Manual*. K&S Ranch, 2012.
[2] G. Xiang *et al.*, "A supervised approach to predict company acquisition with factual and topic features using profiles and news articles on TechCrunch," *Proc. ICWSM*, 2012.
[3] B. Kovacs, G. R. Carroll, and D. W. Lehman, "Authenticity and consumer value ratings," *Organization Science*, vol. 25, no. 2, pp. 458–478, 2014.
[4] L. Breiman, "Random Forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
[5] T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," *Proc. ACM SIGKDD*, pp. 785–794, 2016.
[6] D. H. Wolpert, "Stacked Generalization," *Neural Networks*, vol. 5, no. 2, pp. 241–259, 1992.
[7] M. K. Chaudhary, "Startup Success Prediction Dataset," Kaggle, 2020. [Online]. Available: https://www.kaggle.com/datasets/manishkc06/startup-success-prediction
[8] F. Pedregosa *et al.*, "Scikit-learn: Machine Learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.
[9] A. Géron, *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*, 2nd ed. O'Reilly Media, 2019.